

TaskFlow — Technical Interview Cheat Sheet (Hinglish)

Complete Preparation Guide: Architecture, Security, Database, Code Quality & Tricky Questions

1. Project Elevator Pitch (30–60 Seconds me kaise explain karenin)

Interview me yeh bolna hai:

"TaskFlow ek role-based Team Task Management System hai jo enterprise organizations ko teams structure karne, tasks delegate karne, workflow monitor karne aur in-app collaboration provide karne ke liye design kiya gaya hai.

Maine backend **ASP.NET Core Web API (.NET 8)** aur **Entity Framework Core** with **SQL Server** me banaya hai, jisme **JWT Bearer Authentication** aur **BCrypt password hashing** use ki hai. Frontend **React 18 with Vite** aur **Bootstrap 5** me centralized **Axios** client ke sath banaya hai.

Is project ka core highlight hai iska **strict role authorization layer**: Admin ke paas full organization access hai; Manager sirf apni team ke tasks create aur assign kar sakta hai (outsider assignment strictly blocked hai); aur standard User sirf apne assigned tasks dekh aur update kar sakta hai (tampering via task ID 403 Forbidden return karti hai). Saath hi 17 automated xUnit tests, Docker containerization aur GitHub Actions CI/CD pipeline bhi configured hai."

2. Architecture & Design Principles

Humne **Clean 3-Tier Layered Architecture** follow kiya hai:

- **Controllers**: HTTP requests handle karte hain, routing, model validation aur DTO serialization karte hain.
- **Services & Interfaces**: Saari business rules, role scoping aur notification dispatching yahan hoti hai.
- **Data Layer (ApplicationDbContext)**: EF Core 8 models, database constraints, relationships aur Code-First migrations manage karta hai.
- **Middleware**: Global ExceptionMiddleware jo unhandled exceptions ko catch karke uniform clean JSON return karta hai bina internal stack traces leak kiye.

 **Super Common Interview Question:** "Aapne Repository / Unit of Work pattern alag se kyun nahi banaya?"

Aapka Answer:

"Entity Framework Core ka DbContext internally already **Unit of Work** pattern implement karta hai aur DbSet<T> already **Repository** pattern implement karta hai. Unke upar ek aur custom generic repository layer create karne se abstraction over abstraction ho jata hai aur EF Core ke powerful features jaise .Select() projections, optimized queries aur change tracking restrict ho jate hain. Isliye modern .NET 8 best practices ke mutabiq maine Controller → Service → DbContext architecture choose kiya jo clean, maintainable aur highly testable hai."

3. Database Design & Relations (SQL Server + EF Core)

Entity	Key Columns	Relationships / Constraints
User	Id, Name, Email, PasswordHash, Role, CreatedAt	Email is Unique Indexed. Roles: Admin, Manager, User.
Team	Id, Name, Description, ManagerId, CreatedAt	Foreign key to User (Manager) with DeleteBehavior.Restrict.

Entity	Key Columns	Relationships / Constraints
TeamMember	Id, TeamId, UserId	Composite Unique Index on (TeamId, UserId). Cascades on Team/User delete.
TaskItem (Tasks)	Id, Title, Description, TeamId, AssignedToUserId, CreatedByUserId, Priority, Status, Deadline	FK to Team (Cascade), FK to AssignedToUser (SetNull), FK to CreatedByUser (Restrict).
Comment	Id, TaskId, UserId, Content, CreatedAt	FK to Task (Cascade), FK to User (Restrict).
Notification	Id, UserId, TaskId, Message, Type, IsRead, CreatedAt	FK to User (Cascade), FK to Task (Cascade).

Q: SQL Server me Foreign Key Cycle issue ko kaise handle kiya?

Ans: "SQL Server multiple cascading paths allow nahi karta. Maine OnModelCreating me User foreign keys pe DeleteBehavior.Restrict lagaya aur dependent parent-child (jaise Team → Tasks → Comments) pe DeleteBehavior.Cascade lagaya. Isse circular cascade deadlock completely prevent ho jata hai."

4. Authentication & Security (Deep Dive)

- **Password Hashing:** BCrypt .Net-Next use kiya hai. BCrypt me built-in salt generation aur work-factor (cost) hota hai, jo rainbow table attacks aur brute-force attacks ko prevent karta hai. Plain text passwords kabhi store nahi hote.
- **JWT Structure:** Token me 3 primary claims embed kiye hain:
 - `ClaimTypes.NameIdentifier / userId`
 - `ClaimTypes.Email`
 - `ClaimTypes.Role`
- **Token Expiration:** 24 hours expiry time set hai.
- **Axios Interceptors:**
 - *Request Interceptor:* Har outgoing request ke header me automatically `Authorization: Bearer {token}` attach karta hai.
 - *Response Interceptor:* Agar server **401 Unauthorized** return kare (token expired/invalid), to localStorage clear karke user ko automatically `/login?expired=true` pe redirect karta hai.

5. Role Authorization & Business Rules (Kaise Enforce Kiya)

Is project me role authorization sirf frontend button hide karke nahi, balki **Backend Service Layer me physically enforce** kiya gaya hai:

Rule 1: Manager kisi doosri team ke bande ko task assign nahi kar sakta:

"TaskService me jab Manager task create ya reassign karta hai, to service verify karti hai ki kya target user TeamMembers table me us TeamId ke sath linked hai ya nahi. Agar outsider hai, to backend turant `BadRequestException('Cannot assign a task to a user outside your team')` throw karta hai."

Rule 2: User doosre user ka task ID change karke access ya modify nahi kar sakta (IDOR Prevention):

"Jab User `GET /api/tasks/{id}` ya `PATCH /api/tasks/{id}/status` call karta hai, to service caller ka `currentUserId` extract karti hai aur check karti hai ki `task.AssignedToUserId == currentUserId` hai ya nahi. Agar kisi doosre user ka task ID dala gaya ho, to backend directly **403 Forbidden** return karta hai."

6. Notifications & Dashboard Logic

- **Automated Notifications:**
 1. Jab koi task kisi user ko **assign** hota hai → Assignee ko notification jati hai: `"You have been assigned to task: '{title}'"`.
 2. Jab task ka **status change** hota hai → Related assignee aur creator ko notification jati hai: `"Task '{title}' status changed from 'To Do' to 'In Progress'"`.
- **Role-Tailored Dashboards:**
 - `GET /api/dashboard/admin` → Total Users, Total Teams, Total Tasks, To Do, In Progress, Done, Overdue, High Priority.
 - `GET /api/dashboard/manager` → Manager ki managed team ke statistics aur team member count.
 - `GET /api/dashboard/user` → User ke assigned tasks ke statistics aur upcoming deadlines.

7. Testing Strategy (xUnit + Moq + InMemory DB)

17 Automated Tests banaye hain jo 100% pass ho rahe hain:

- AuthServiceTests: Valid registration, duplicate email rejection (400), login validation, wrong password rejection.
- TaskServiceTests: Manager task creation inside team vs outside team rejection, User creation blocked (403), User status update on assigned vs unassigned task rejection (403), Manager assignment to team member vs outsider rejection.
- AuthorizationTests: JWT claim token generation aur ClaimsPrincipal extraction.

"Maine InMemory Database isliye use kiya taaki unit tests fast execute hon, test cases isolated rahen aur CI/CD pipeline me external SQL Server pe depend na hona pade."

8. Top 10 Tricky Interview Questions & Ready Hinglish Answers

1. DTOs (Data Transfer Objects) kyun use kiye? Direct EF Core Entities kyun nahi return kiye?

Ans: "Teen primary reasons hain: 1) **Security & Over-Posting:** Direct entity lena se malicious users internal columns (jaise Role ya PasswordHash) manipulate kar sakte hain. 2) **Circular References:** Entities me bidirectional navigation properties hoti hain (User → Team → User) jo JSON serializer me infinite loop error deti hain. 3) **Payload Optimization:** Client ko sirf wahi fields bhejna jo zaroori hain, sensitive columns hide karke."

2. Global Exception Handling kaise implement ki hai?

Ans: "Maine custom `ExceptionHandler` implement kiya hai jo HTTP pipeline me sabse pehle invoke hota hai. Agar koi service exception throw karti hai (e.g. `NotFoundException`, `ForbiddenException`, `BadRequestException`), to middleware use catch karta hai, `ILogger` se server console me log karta hai, aur client ko clean standardized JSON response `{ success: false, message: '...' }` with proper HTTP status code (404, 403, 400, 500) return karta hai bina stack traces expose kiye."

3. Entity Framework Core me N+1 Query Problem ko kaise prevent kiya?

Ans: "Maine Eager Loading use ki hai via `.Include()` aur `.ThenInclude()` jisse EF Core single SQL JOIN query generate karta hai loop me alag-alag queries run karne ke bajaye. Saath hi read-only queries me `.AsNoTracking()` use kiya hai jo EF change tracker ki overhead ko eliminate karta hai aur query performance 3x fast karta hai."

4. React me JWT token expiration kaise handle kiya hai?

Ans: "Axios response interceptor configure kiya hai. Jab bhi backend se **401 Unauthorized** status code aata hai (matlab token expire ho chuka hai ya invalid hai), interceptor automatically `localStorage` se token aur user details clear karta hai aur window location ko `/login?expired=true` pe redirect kar deta hai."

5. Swagger me JWT Bearer authentication kaise setup kiya?

Ans: "Program.cs me `AddSwaggerGen` ke andar `AddSecurityDefinition` define kiya jisme `SecuritySchemeType.Http` aur Scheme 'Bearer' di. Saath hi `AddSecurityRequirement` lagaya. Isse Swagger UI ke top-right me 'Authorize' button aa jata hai jahan login API se mila JWT paste karke protected endpoints directly test kiye ja sakte hain."

6. Docker me multi-container setup kaise work karta hai?

Ans: "docker-compose.yml me 3 services hain: `sqlserver` (MSSQL 2022 Linux image), `backend` (Multi-stage .NET 8 build on port 5000), aur `frontend` (Multi-stage Node build & Nginx alpine on port 3000). Teeno ek common bridge network pe communicate karte hain aur SQL Server ka data named volume me persist hota hai."

7. CI/CD pipeline me kya setup kiya hai?

Ans: "GitHub Actions workflow (`.github/workflows/ci.yml`) create kiya hai jo har git push aur pull request par automatically .NET 8 restore & build karta hai, 17 xUnit tests run karta hai, aur React frontend build (`npm run build`) verify karta hai. Agar test fail ho to PR merge nahi ho sakti."

8. Task statuses aur priorities kya hain?

Ans: "Statuses: **To Do**, **In Progress**, **Done**. Priorities: **Low**, **Medium**, **High**, **Urgent**. Database me inke regular expression validation DTOs me lage hain."

9. Sample Credentials & Demonstration Flow

Role	Email	Password	Key Features to Demonstrate
Admin	admin@example.com	Password123!	Organization-wide overview metrics, team creation, full task oversight, user directory.
Manager	manager@example.com	Password123!	Platform Engineering team dashboard, creating tasks for team members, adding members.
User	user@example.com	Password123!	Personal task workspace, inline status updates (To Do → Done), collaborative comments, notification bell.